IT-TAGE 2026 · FRANKFURT · 9 DECEMBER 2026

Playwright
Beyond the Happy Path

E2E tests, accessibility and MCP in practice. Sven Hennessen · vensas GmbH

vensas

About me



- **Almost 20 years** of software development and architecture
- **Co-founder and co-managing director** of vensas GmbH, founded in 2023
- Consulting and development of **business software** for medium-sized companies
- Away from the computer — **hiking**, **cycling** or the **racing simulator**

Sven Hennessen
vensas GmbH

The demo app: Deploy or Die

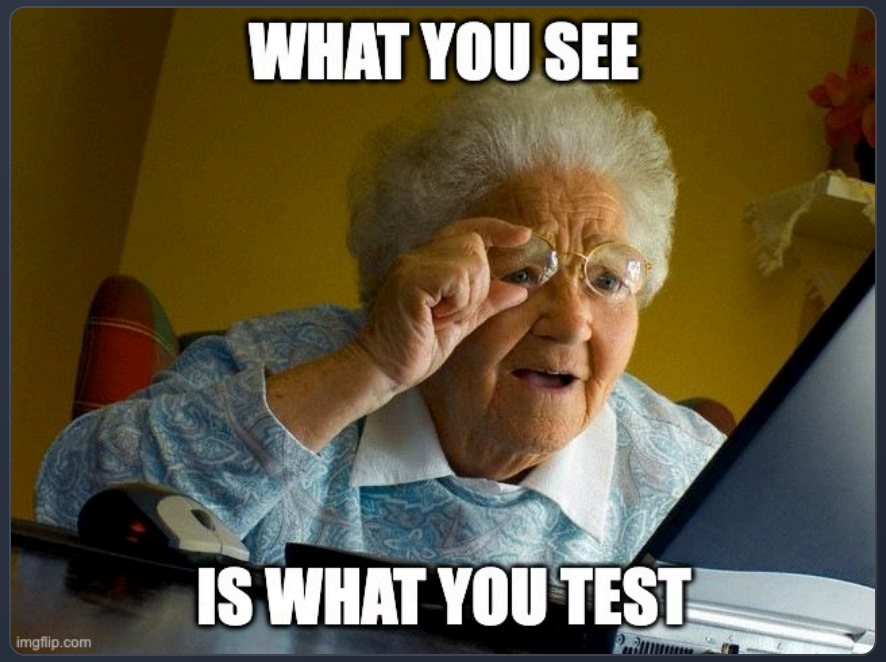
A deployment game. Each deployment has a 50% success rate. After a deployment you must wait 30 seconds.

Frontend	React 19, PrimeReact, Rspack
Backend	ASP.NET Core minimal API on .NET 10, PostgreSQL 16
Tests	Playwright 1.63 in TypeScript, 1.62 in C#

UI mode

What you get

- Watch mode — the test runs again after each change
- Time travel — see the DOM at each step
- Pick locator — click an element, get the locator
- Network, console and source in one window



FROM A REAL PROJECT

Keep the tests in their own folder

playwright.config.ts

```
// We did this. Playwright then read all of src/ at each start.
testDir: './src',

// Keep the default. The tests have their own folder.
testDir: './tests',
```

We made this change. Then we made it again in the other direction.

What it cost us

CI	The test job became slow, then it stopped with a timeout.
UI mode	The start took some minutes, but not each time.

Recommendation

Keep the tests out of the source folder.

Traces

playwright.config.ts

```ts
use: {
  trace: {
    mode: 'on',
    snapshots: { dom: true, aria: true, screen: true },
  },
  screenshot: 'only-on-failure',
  video: 'retain-on-failure',
}
```

New in 1.63: the trace also records the ARIA tree and the screen.

A trace contains

- DOM, ARIA tree and screen for each step
- All network requests and answers
- Console output and source code

FROM A REAL PROJECT

Make the trace available in CI

A test can fail in CI and pass on your machine. Without the trace you can only guess.

.github/workflows/e2e.yml

```yaml
- name: Upload the report
  uses: actions/upload-artifact@v7
  if: always()
  with:
    name: playwright-report
    path: playwright-report/
    retention-days: 7

- name: Upload traces and videos
  uses: actions/upload-artifact@v7
  if: failure()
  with:
    name: playwright-traces
    path: test-results/
    retention-days: 7
```

The report goes up after each run. The trace and the video go up only after a failure.

Recommendation

Upload the report after each run, and the traces after each failure.

FROM A REAL PROJECT

Keep the test pyramid

UI mode and agents make an E2E test look cheap. It is not: it costs more to write and to maintain than a unit test.

From a team that I mentored

Most tests were Playwright tests. React Testing Library was sufficient for 80% of them. The suite grew, and the CI runners became unreliable.

What I take from this

Many — **Unit and component tests. They are fast and cheap.**

Few — **E2E tests. Use them for the important paths.**

> An unreliable E2E test is worse than no E2E test.

Recommendation

Find the E2E tests that a unit or a component test can replace.

Mock the API

tests/deploy-failure.spec.ts

```
// Set the route before you go to the page.
await page.route('**/api/deploy', async (route) => {
  await route.fulfill({
    status: 200,
    contentType: 'application/json',
    body: JSON.stringify({
      success: false,
      message: '💥 BOOM! The database just went on vacation.',
      deployedBy: 'Unlucky Developer',
    }),
  });
});

await page.goto('/');
```

You control the answer. Thus you can test the error path, which occurs in only 50% of the runs.

FROM A REAL PROJECT

Mock or real — how I choose

Both are correct, and both add value. The complexity of the project decides.

Three questions before I decide

Services	How many services does the scenario touch?
Contract	How complex is the API contract?
Test	How complex is the test itself?

Mock or real

Mock	**Fast and stable. Blind to the backend.**
Real	**Finds integration faults. Slow.**



Recommendation

The demo app has one service and a small contract. Thus it has 19 mocked tests and 4 integration tests.

FROM A REAL PROJECT

Start the dependencies with Compose

First I started each container from the test code. It was flaky, on my machine and in CI.

What broke

- The start order and the readiness check
- The network between the containers and the port mapping
- Containers that stayed after the run

tests/docker-compose.integration.yml

```
postgres:
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U postgres"]
backend:
  depends_on:
    postgres:
      condition: service_healthy
```

depends_on with a healthcheck replaces the wait code in the test.

Recommendation

Start the containers one time in a global setup, not for each test.

FROM A REAL PROJECT

The E2E tests are not only a frontend job

Teams put the E2E tests in the frontend pipeline only. But a test against a real backend also tests the backend.

What happened to a team that I mentored

The team released an API that was not compatible. Nobody ran the E2E tests before the release.

What I recommend

- Run the tests after each frontend build
- Run them after each backend build too
- Or run them nightly — your test plan decides

Recommendation

Find out where your E2E tests run today.

More than one page at a time

tests/multiple-pages.spec.ts

```
test('two users deploy at the same time', async ({ browser }) => {
  const alice = await browser.newContext();
  const bob = await browser.newContext();
  await Promise.all([mockDeploy(alice), mockDeploy(bob)]);

  const [alicePage, bobPage] = await Promise.all([alice.newPage(), bob.newPage()]);
  await Promise.all([alicePage.goto('/'), bobPage.goto('/')]);

  await Promise.all([deploy(alicePage, 'Alice'), deploy(bobPage, 'Bob')]);

  await expect(alicePage.getByText(/Deployed by: Alice/i)).toBeVisible();
  await expect(bobPage.getByText(/Deployed by: Bob/i)).toBeVisible();
});
```

Each context has its own cookies and storage. Promise.all lets both users click at the same time.

Pages and contexts

Page	One tab. context.newPage()
Context	One user. browser.newContext() — the routes apply to all its pages
New tab	context.waitForEvent('page') — start to wait before the click

Accessibility as a gate

tests/accessibility.spec.ts

```
const results = await new AxeBuilder({ page })
  .withTags(['wcag2a', 'wcag2aa', 'wcag21a', 'wcag21aa', 'wcag22aa'])
  .analyze();

await testInfo.attach('axe-results.json', {
  body: JSON.stringify(results.violations, null, 2),
  contentType: 'application/json',
});

expect(results.violations.map((v) => ({ rule: v.id, impact: v.impact }))).toEqual([]);
```

The European Accessibility Act applies since June 2025. Thus this check belongs in the pipeline.

Expectation management

axe-core finds 25% to 57% of the defects*.

Recommendation: test the rest by hand, with the keyboard and a screen reader.

* Deque, 57% · UK Government Digital Service tool audit, 30–40% · Karl Groves, 25–33%

The same test in C#

DeployTests.cs

```
public class DeployTests : PageTest
{
    [Fact]
    public async Task Cooldown_BlocksTheSecondDeployment()
    {
        await Page.Clock.InstallAsync();
        await Page.GotoAsync("http://localhost:3000");

        var deploy = Page.GetByRole(AriaRole.Button,
            new() { NameRegex = new Regex("Deploy Now") });
        await deploy.ClickAsync();

        // Move the time forward instead of a wait of 30 seconds.
        await Page.Clock.FastForwardAsync("00:30");
        await Expect(deploy).ToBeEnabledAsync();
    }
}
```

The same API, with .NET names. Your test runner is xUnit, NUnit or MSTest.

Same	**Locators, Clock API, routes, traces, codegen, axe-core**
Node only	**UI mode, test locks, isolated retries, Test Agents**

FROM A REAL PROJECT

Where you must hold the reins

A generated test can look correct and be worthless

Typical defects: a hard coded wait, a CSS selector instead of a role, and an assertion that is always true.

```
// Generated — and wrong.
await page.waitForTimeout(3000);
await page.click('.p-button.p-component.deploy-button');
await expect(page.locator('body')).toBeVisible();
```

The rule that works for us

Agent	**Explores and writes the first draft.**
Human	**Reviews, then puts it in the pipeline.**

Recommendation

Let an agent write the first draft, not `playwright codegen`. The agent reads the accessibility tree and uses roles. Review each draft.

Demo: an agent tests the app over MCP

The MCP server gives the agent the accessibility tree, not pixels. It is part of Playwright since 1.62.

.mcp.json

```
{
  "mcpServers": {
    "playwright": { "command": "npx", "args": ["playwright", "mcp"] }
  }
}
```

STEP 1
Prompt
Test the error path of the deploy form.

→

STEP 2
Snapshot
browser_snapshot gives the ARIA tree, with a ref for each element.

→

STEP 3
Act
browser_type and browser_click use the refs.

→

STEP 4
Result
A spec with role based locators. You review it.

playwright-cli: the agent runs code

One snippet instead of ten tool calls. The snippet has the full Playwright API.

agent-snippets/cooldown.js

```
async page => {
  await page.route('**/api/deploy', route =>
    route.fulfill({ json: { success: false, message: 'BOOM' } }));
  await page.clock.install();
  await page.reload();
  await page.getByLabel(/Who dares to deploy/i).fill('Agent');
  await page.getByRole('button', { name: /Deploy Now/i }).click();
  await page.getByText('BOOM').waitFor();
  const during = await page.locator('.deploy-button').textContent();
  await page.clock.runFor('00:30');
  const after = await page.locator('.deploy-button').textContent();
  return { during, after };
}
```

Result: {"during":"Next deploy in 30s","after":"Deploy Now 🚀 "}

```
playwright-cli open http://localhost:3000
playwright-cli run-code \
  --filename=agent-snippets/cooldown.js
```

Tokens	Commands only, no tool schema in the context
Reach	Routes, clock and more pages in one call
Repeat	The agent keeps the file and runs it again
Attach	The agent uses your open Chrome, with your login

TL;DR

1. **Write few E2E tests. The test pyramid still holds.**
   An unreliable E2E test is worse than no E2E test.
2. **An E2E test with a real backend is not a frontend job.**
   Run it after each backend build too, or nightly.
3. **Make accessibility a gate on day one.**
   axe-core is a few lines in the test. The European Accessibility Act applies since June 2025.
4. **AI: let agents drive the browser with the MCP server and playwright-cli.**
   Use them to explore and to automate. A human reviews each test that they write.

That's all folks!

THANK YOU

Questions?

Demo project and slides available at https://github.com/vensas/playwright-2026-talk

https://www.linkedin.com/in/svenhennessen/

kontakt@vensas.de

vensas.de

Slides & Demo